API 文件前置作業

目錄

1.1 API 認證

1.2 API 共通規範

1.3 狀態碼

1.4 轉真人 Webhook 事件

1.5 客戶端工具（Client Tools）

1.6 訊息 Webhook

1.7 偵測新訊息

1.8 解析器自動偵測結果

API 認證

MaiAgent 使用 API 金鑰進行身分驗證。

身分認證方式

所有 API 請求必須在 HTTP Header 中包含 API Key，格式如下：

```
const headers = {
  "Authorization": "Api-Key YOUR_API_KEY_HERE",
  "Content-Type": "application/json"
}
```

標頭的前綴是 `Api-Key`，不是 `Bearer`。金鑰綁定哪位成員、權限怎麼算、什麼情況會失效等規則，見 API 共通規範：認證。

如何獲取 API 金鑰？

API 金鑰有兩種取得方式，兩種金鑰的呼叫方式相同：

成員自建：組織內的有效成員都可以在後台右上角頭像 → 帳號 → API 金鑰 為自己建立金鑰，不需要額外權限。

管理者代發：具備「API Key 管理權限」的管理者，可以在 組織設定 → API Key 管理 替指定成員簽發金鑰，並設定到期時間與每分鐘請求數（RPM）。

操作步驟見使用者手冊 API Key 管理：成員自行建立金鑰、管理者建立 API Key。

> 安全提醒：請妥善保管您的 API 金鑰，不要在公開場所分享 金鑰格式說明：API 金鑰由前綴與密鑰兩段英數字組成，中間以 . 相連，呼叫時請完整帶入

> 完整金鑰只會在建立成功時顯示一次。之後的列表僅顯示部分字元，無法再次查看或還原完整金鑰；若未保存，請建立新金鑰、更新使用該金鑰的服務，再撤銷舊金鑰。

找不到舊的系統簽發金鑰

個人帳號的 API 金鑰頁面只列出您自行建立的金鑰。首次加入組織（包含透過 AD 首次登入）時，系統可能會自動簽發一把無到期日的金鑰，但不會提供其完整值，因此無法取得該金鑰來呼叫 API。若要呼叫 API，請在個人頁面自行建立新金鑰，並在建立成功時立即保存完整值。

系統自動簽發或由管理者代為簽發的金鑰，只有具有組織金鑰管理權限的成員可從 組織金鑰管理查看其擁有者、前綴、簽發來源、狀態與到期日，但該頁不會顯示完整值。若您沒有此權限，可請組織擁有者或具有組織金鑰管理權限的成員協助確認或撤銷舊金鑰。

API 金鑰的權限範圍

以 API 金鑰呼叫 API 時，權限與金鑰擁有者在該組織的角色權限相同；缺少所需權限時，API 會回傳 403 Forbidden。

成員（members）與角色（groups）的寫入操作都需要子權限 組織概覽權限（位於 組織權限 之下），組織擁有者則不受此限。適用的操作包含：

新增成員：`POST /api/v1/organizations/{organizationPk}/members/`
移除成員：`DELETE /api/v1/organizations/{organizationPk}/members/{id}`
調整成員所屬角色：`POST /api/v1/organizations/{organizationPk}/members/{id}/update-groups/`
建立、更新、刪除角色：`POST`
`PUT`
`PATCH`
`DELETE /api/v1/organizations/{organizationPk}/groups/…`
批量新增或移除角色成員、分配或移除角色的 AI 助理、收件匣、知識庫與資料庫：`/api/v1/organizations/{organizationPk}/groups/{groupPk}/group-*…`
其他成員與角色的寫入操作（例如批次匯入成員）同樣適用

> 權限以子項判斷：角色設定中父層 組織權限 未勾，不代表其下的 組織概覽權限 未勾。排查 403 時，請到後台 組織設定 → 角色 展開 組織權限，確認金鑰擁有者所屬角色是否勾選 組織概覽權限。

錯誤處理

當認證錯誤時，API 會回傳 401 Unauthorized 的 HTTP 錯誤代碼：

API Key 缺失、格式錯誤或無效

沒帶金鑰、標頭格式不對，或金鑰無效（不存在、已到期、已停用、已撤銷，或綁定的成員已停用、已移出組織），都回同樣的內容，不會說明是哪一種原因：

```
{
  "detail": "Authentication credentials were not provided."
}
```

修復建議：檢查請求標頭是否為 `Authorization: Api-Key YOUR_API_KEY`，並確認 API Key 完整、沒有多餘的空格或字元。格式正確仍失敗時，請到 API Key 管理 確認金鑰不是已停用、已過期或已撤銷，以及綁定的成員仍是啟用狀態。

把 API Key 放在 `Bearer` 後面

平台會把它當成登入權杖驗證，回應內容不同：

```
{
  "detail": "Given token not valid for any token type",
  "code": "token_not_valid",
  "messages": [ … ]
}
```

修復建議：把前綴改成 `Api-Key`。

`401` 回應會帶 `WWW-Authenticate` 標頭，多數端點的值是 `Bearer realm="api"`，請以狀態碼判斷，不要依這個標頭決定要帶哪一種憑證，說明見 API 共通規範：認證失敗的回應。金鑰有效但權限不足時回 `403`，見上方 API 金鑰的權限範圍。

API 共通規範

本頁整理所有平台 API 共用的規則：網址、認證、限流、錯誤回應、分頁、欄位格式與上傳上限。各端點的欄位與範例見左側「API 端點說明文件」，本頁的規則適用於全部端點，端點頁不再重複。建議在開始寫程式前讀過一次。

> 建立與管理 API Key 的畫面操作見使用者手冊 API Key 管理；查呼叫紀錄見 API 呼叫紀錄。 從零開始串接的步驟見技術人員手冊 快速上手。

網址與版本前綴

SaaS 的 API 網址是 https://api.maiagent.ai/api/v1/；地端與私有雲部署請換成貴公司的平台網域。

`/api/` 與 `/api/v1/` 掛的是同一套端點，路徑、欄位與權限都相同。
AI Gateway（OpenAI 相容端點）與 SCIM **只在 `/api/v1/` 底下**。
新的整合請一律使用 `/api/v1/`。
端點路徑以
結尾。少了結尾的
會收到 `301` 轉址到加上
的網址；部分 HTTP 用戶端跟隨轉址時會把 `POST` 改成 `GET`，請直接呼叫帶
的網址。

認證

伺服器對伺服器的呼叫使用 API Key，放在 `Authorization` 標頭：

```
Authorization: Api-Key <您的 API Key>
```

金鑰由前綴與密鑰兩段英數字組成，中間以 `.` 相連，請完整帶入。

規則	說明
金鑰綁定一位成員	呼叫時的組織就是這位成員所屬的組織，請求不需要另外帶組織 ID
權限等於該成員的角色	沒有依端點細分的 scope；成員能做的事，金鑰都能做。缺少權限回 `403`，見 API 金鑰的權限範圍
成員停用即失效	成員被停用或移出組織後，這把金鑰立即無法使用

規則	說明
到期、停用、撤銷	和不存在的金鑰一樣回 `401`，回應不會說明原因
完整金鑰只顯示一次	建立後只會顯示部分字元，遺失請重新建立

金鑰的建立與管理入口見 API 認證方式。

認證失敗的回應

沒帶金鑰、標頭格式不對，或金鑰無效（不存在、已到期、已停用、已撤銷，或綁定的成員已停用、已移出組織）都回 `401`，內容相同：

```
{"detail": "Authentication credentials were not provided."}
```

金鑰無效時，訊息同樣是「沒有提供認證資訊」：平台把認不得的金鑰當成沒有帶憑證，不會說明是哪一種原因。

回應帶 `WWW-Authenticate` 標頭。多數端點的值是 `Bearer realm="api"`（這些端點也接受 OAuth 存取權杖）；只接受 API Key 的端點（例如下方的 `api-key-context`）是 `Api-Key`。請以 `401` 狀態碼判斷，不要依這個標頭的值決定要帶哪一種憑證。

把 API Key 放在 `Bearer` 後面，平台會把它當成登入權杖驗證，回 `401` 與不同的內容：

```
{"detail": "Given token not valid for any token type", "code": "token_not_valid",
"messages": [ … ]}
```

收到這個回應時，請把標頭改成 `Api-Key <金鑰>`。

> 整合用金鑰的建議：建一位專用的整合成員，只給它需要的角色權限，再用它的身分簽發金鑰並設定到期時間。綁在個人帳號上的金鑰，會在這位員工離職、帳號被停用時一起失效，整合也跟著中斷。API Key 請只放在伺服器端：平台 API 只對允許的網域開放跨來源（CORS）呼叫，也不應該讓金鑰出現在瀏覽器裡。

確認金鑰所屬的組織

要在自家閘道先確認金鑰屬於哪個組織，可以呼叫 `GET /api/v1/users/api-key-context/`，回應 `{"organizationId": "…"}`。這支端點只接受 API Key，不計入金鑰的 RPM。

其他憑證

讓 MCP 客戶端以成員身分連線，見技術人員手冊 MaiAgent 作為 OAuth 授權伺服器；在自家系統嵌入 MaiGPT 的成員權杖，見 串接 MaiGPT：成員身分嵌入。

速率限制

平台 API **沒有全域的預設上限**。會限流的只有下表這些：

範圍	上限	計算方式	誰可以調整
每把 API Key 的請求數（RPM）	管理者設定，未設定時不限	每把金鑰、每 60 秒	組織的 API Key 管理
AI Gateway 的 Token 數（TPM）	管理者設定，未設定時不限	每把金鑰、每分鐘，只算 Gateway 推論	同上，見 AI Gateway 的限制
SCIM 端點	100 次／分鐘	每個組織	固定
`web-chats/{id}/setup-contact-credentials/`	30 次／分鐘	每個來源 IP	固定
免登入的計價查詢（`credit/items/`、`credit/pricing-rules/`）	60 次／分鐘	每個來源 IP	固定
`auth/login/`	預設 10 次／分鐘	每個來源 IP	平台設定
不帶憑證的 Web Chat 請求（接續對話、上傳附件等）	10～30 次／分鐘，依端點而定	每個來源 IP	固定

金鑰的 RPM 作用在這把金鑰能呼叫的所有平台 API，不只 AI Gateway。端點本身另有上限時（例如表中的固定上限），用 API Key 呼叫會同時受兩道上限限制：兩邊各自計數，哪一道先達到就先回 `429`，`Retry-After` 取需要等待較久的那一道。例外有兩種：確認金鑰所屬的組織 的端點不計入金鑰的 RPM；免登入的計價查詢（`credit/items/`、`credit/pricing-rules/`）不辨識憑證，帶著 API Key 呼叫也只受每個來源 IP 的上限，不計入金鑰的 RPM。

OAuth 存取權杖與後台登入的請求不受金鑰 RPM 限制。

超過上限回 `429`，並帶 `Retry-After` 標頭（秒數）。請依這個秒數退避後再重試，不要立刻重送。

```
{"detail": "Request was throttled. Expected available in 42 seconds."}
```

錯誤回應

平台 API 沒有統一的錯誤碼表。處理錯誤時請依 **HTTP 狀態碼**判斷，回應帶 `code` 時再依 `code` 細分；**不要比對 `detail` 文字**：文字會隨 `Accept-Language` 翻譯，也可能調整措辭。

回應內容的形狀

形狀	範例	常見於
`detail` 物件	`{"detail": "You do not have permission to perform this action."}`	認證、權限、找不到、限流、方案與 Credits；少數會多帶 `code` 或其他欄位
欄位物件	`{"name": ["This field is required."]}`	欄位驗證失敗；key 是 camelCase 欄位名，不屬於單一欄位的錯誤放在 `nonFieldErrors`
字串陣列	`["…"]`	少數沒有對應欄位的驗證錯誤

例外情況：

AI Gateway 的 OpenAI 相容端點使用 OpenAI 格式的 `error` 物件，見 AI Gateway 錯誤格式。

SCIM 端點的請求內容錯誤、找不到與衝突，依 SCIM 2.0 規格回應（`schemas`、`status`、`detail`）。

網址不存在（路徑打錯、少了資源 ID）時回 `404`，內容是 HTML，不是 JSON；`5xx` 回應也不保證是 JSON。解析回應前請先確認 `Content-Type`。

狀態碼

狀態碼	意義	建議處理
`200` ／ `201` ／ `204`	成功；`204` 沒有回應內容，常見於刪除	—
`202`	已受理但尚未完成，例如同步回覆時附件還在建立索引，回應 `status: "pending_attachments"`	稍後重試，見 不會進入串流的情況
`301`	路徑少了結尾的	改用帶 的網址
`400`	欄位驗證失敗；資源 ID 不是有效的 UUID 時，多數端點回 `{"detail": "Invalid UUID format."}`	修正請求，不要重試
`401`	沒帶憑證或憑證無效，見 認證失敗的回應	檢查金鑰與標頭格式，不要重試
`402`	方案、Credits 或額度不足，見 下方說明	通知組織管理者
`403`	角色權限不足、組織未開通該功能、組織已停用，或字數計費方案的用量已達上限	調整角色權限或洽窗口
`404`	資源不存在、不屬於金鑰所屬的組織，或網址打錯	確認 ID、金鑰所屬組織與路徑

狀態碼	意義	建議處理
`409`	與資源目前的狀態衝突，例如同一件工作已在執行中、要刪除的資源仍被引用	讀回最新狀態再決定，不要原樣重送
`429`	超過速率限制	依 `Retry-After` 退避
`5xx`	平台或上游服務錯誤	指數退避後重試

`409` 的回應是 `detail` 物件，部分端點會附上處理所需的欄位，例如知識庫已有 FAQ 匯入在執行時：

```
{"detail": "An FAQ import is already running for this knowledge base.", "jobId": "…"}
```

402：方案、Credits 或額度不足

情況	回應內容	有沒有 `code`
組織沒有有效的付費方案	`{"detail": "Your organization has no active paid plan. …", "code": "no_active_plan"}`	有，`no_active_plan`
組織 Credits 不足	`{"detail": "Insufficient credits …"}`，文字依端點而異	沒有
成員的個人額度用完或未分配，或所屬角色的額度用完	`{"detail": "Personal credit quota exhausted. …"}`，文字依情況而異	沒有

只有「沒有有效的付費方案」會帶 `code`。Credits 不足與額度用完的回應只有 `detail`，目前無法只從回應內容分辨是組織 Credits 不足，還是個人或角色額度用完，請組織管理者到後台確認。

Credits 與額度在會產生用量的寫入請求檢查，例如送出訊息、知識庫檔案上傳與搜尋、網頁爬取；查詢類的 `GET` 請求不會因此回 `402`。

AI Gateway 的 OpenAI 相容端點回 OpenAI 格式：`{"error": {"type": "billing_error", "code": "insufficient_credits", …}}`，沒有有效方案時也是這個 `code`。

平台未啟用 Credits 計費時（部分地端部署），不會出現這類 `402`。

分頁

多數列表端點用頁碼分頁：

參數	說明
`page`	頁碼，從 1 開始；超出範圍回 `404` `{"detail": "Invalid page."}`
`page_size`（也可寫 `pageSize`）	每頁筆數，預設 20、上限 100；超過 100 會自動截成 100，不會報錯

回應格式為 `{"count": …, "next": "…", "previous": "…", "results": [ … ]}`，取下一頁時直接使用 `next` 的網址，沒有下一頁時為 `null`。

另外兩種情況：

游標分頁：對話列表（`GET /api/v1/conversations/`）與訊息列表（`GET /api/v1/messages/`）用 `cursor` 參數，由新到舊翻頁，以 `next` 取更早的資料。訊息列表的回應帶 `count`，對話列表沒有；訊息列表每一頁內的訊息依時間由舊到新排列。

不分頁：部分清單直接回傳陣列，例如 `GET /api/v1/credit/items/`。

各端點屬於哪一種，以該端點頁的回應範例為準。

欄位格式

命名：回應的 key 一律是 camelCase。請求的 JSON 與查詢參數可以用 camelCase，snake_case 也接受。少數自由格式的欄位內容會原樣保留、不轉換大小寫，例如 JSON Schema、自訂 HTTP 標頭與 metadata。

時間：多數時間欄位輸出為 Unix 時間戳記的毫秒字串，例如 `"1759536000000"`，精度到秒（最後三位固定是 `000`）。少數欄位使用 ISO 8601 字串，以該端點頁為準。

小數：金額、單價等小數欄位以字串輸出，例如 `"316.8000000000"`，請用十進位型別處理，避免浮點誤差。

ID：資源 ID 多為 UUID。

語系：錯誤訊息會依 `Accept-Language` 標頭翻譯（例如 `ja`），沒有翻譯的訊息維持英文，請勿依賴訊息文字。

上傳大小上限

上傳上限由平台設定，可能調整，地端部署也可能不同。請以 `GET /api/v1/allowed-file-specs/` 的回應為準：這支端點不需認證，依用途列出允許的副檔名與 `maxSize`（位元組）。2026 年 10 月正式區的值如下：

用途	上限
知識庫檔案	100 MB
對話附件：文件、試算表、簡報與音檔	10 MB
對話附件：圖片	5 MB

用途	上限
會議記錄音檔	100 MB

以 Presigned 方式上傳時，取得上傳網址的 `POST /api/v1/upload-presigned-url/` 就會依 `fileSize` 與副檔名檢查，超過上限或類型不允許回 `400`。流程見技術人員手冊 Presigned 檔案上傳模式。

知識庫的中繼資料、標籤與 FAQ 的 Excel 匯入，共用知識庫檔案的上限。

常見問題

- ▶ 金鑰明明是對的，卻一直回 401？
- ▶ 可以限制某把金鑰只能呼叫特定 API 嗎？
- ▶ 為什麼 `/api/` 開頭的範例和 `/api/v1/` 都能用？

狀態碼

在透過 API 送出 HTTP 請求時，可使用以下狀態碼查看 API 請求成功與否：

狀態碼說明

正確代碼

狀態碼	說明
200	請求成功
201	資源成功建立
204	請求成功，沒有回應內容（常見於刪除）

正確回應範例

> 此處為內容示範，不同端點的回傳格式內容可能有差異，請以該端點的說明文件參考為主

```
{
  "count": "integer",
  "next": "string|null",
  "previous": "string|null",
  "results": [
    // 結果回傳內容
  ]
}
```

錯誤代碼

狀態碼	說明
400	請求格式錯誤
401	認證失敗
402	方案、Credits 或額度不足，見 402 回應
403	權限不足

狀態碼	說明
404	資源不存在
409	與資源目前的狀態衝突，見 409 回應
429	超過速率限制，見 429 回應
500	伺服器錯誤

> 錯誤回應的內容主要有三種形狀：detail 物件（例如 {"detail": "…"}，少數會多帶 code）、欄位物件（key 是 camelCase 欄位名）、字串陣列；AI Gateway 的 OpenAI 相容端點則使用 OpenAI 格式的 error 物件。處理時請依狀態碼與 code 判斷，不要比對 detail 文字。各狀態碼的完整說明與處理建議見 API 共通規範：錯誤回應。

錯誤回應範例

400 - 請求錯誤：必填欄位缺失

```
{
  "files": ["This field is required."],
  "filename": ["This field is required."],
  "file": ["This field is required."]
}
```

402 - 方案、Credits 或額度不足

`402` 有三種情況，只有「沒有有效的付費方案」會帶 `code`：

情況	`code`
組織沒有有效的付費方案	`no_active_plan`
組織 Credits 不足	無
成員的個人額度用完或未分配，或所屬角色的額度用完	無

```
{
  "detail": "Your organization has no active paid plan. Please upgrade to
continue using the platform.",
  "code": "no_active_plan"
}
```

```
{
  "detail": "Insufficient credits for KB search. Please top up your account."
}
```

Credits 不足與額度用完的回應只有 `detail`，文字依端點與語系而異，無法從回應內容分辨是哪一種，請組織管理者到後台確認。AI Gateway 端點的格式不同，見 API 共通規範：402。

409 - 與資源目前的狀態衝突

請求與資源目前的狀態衝突，例如同一件工作已在執行中，或要刪除的資源仍被其他資源引用。回應是 `detail` 物件，部分端點會附上處理所需的欄位：

```
{
  "detail": "An FAQ import is already running for this knowledge base.",
  "jobId": "…"
}
```

請先讀回資源的最新狀態再決定下一步，不要原樣重送。

429 - 超過速率限制

超過 API Key 的每分鐘請求數（RPM）或端點本身的固定上限時回 `429`，並帶 `Retry-After` 標頭（秒數）：

```
{
  "detail": "Request was throttled. Expected available in 42 seconds."
}
```

請依 `Retry-After` 的秒數退避後再重試，不要立刻重送。哪些範圍會限流見 API 共通規範：速率限制。

轉真人 Webhook 事件

對話進入轉真人流程的每個階段——AI 判定需要真人、指派給客服、轉回 AI、排隊逾時——MaiAgent 都會以 HTTPS POST 推送一筆事件到你設定的端點，讓派工、告警、CRM 或你自己的客服系統自動接手，不需輪詢。事件名稱描述轉真人的狀態轉移，是固定的公開契約；每個事件在對話事件紀錄 API（`GET /api/v1/conversations/{id}/events/`）都有對應的 `event_type`（見下表），用於對帳或改以輪詢取得同一份資訊。

啟用方式

由客戶的管理者或串接人員在後台自助完成，毋需工程協助：

後台「客服對話」→「對話平台」→ 編輯要串接的對話平台 →「客服轉真人設定」→「通知」分頁
開啟「Webhook 通知（系統串接）」開關，填入接收端網址（僅接受 `https://` 開頭）
（選填但建議）填入簽章密鑰，啟用來源驗證（見下方「簽章驗證」）

Webhook 與通知中心、Email 是三個獨立開關，互不影響；預設關閉、逐收件匣啟用。此通道與收件匣既有的「訊息 Webhook」（訊息收送）互不相干。

事件種類

event	事件紀錄 API 的 `event_type`	何時送出	備註
`handoff.requested`	`transfer_triggered`	AI 判定需要真人：對話進入排隊、AI 停止自動回覆	每次轉真人流程的第一筆事件；`assignee` 為 `null`。真人客服在你自己的系統時，這就是接手的訊號
`handoff.assigned`	`human_assigned`	對話被指派給 MaiAgent 後台的客服成員（含手動指派、自行認領、轉接、自動指派）	`assignee` 為被指派客服；轉接給另一位客服會再送一筆（事件 `id` 不同）
`handoff.returned_to_ai`	`returned_to_ai`	客服在後台按「轉回 AI Agent」或透過 `return-`	`assignee` 為 `null`（對話已無負責客服）

event	事件紀錄 API 的 `event_type`	何時送出	備註
		`to-ai` API 將對話交回 AI	
`handoff.queue_timeout`	`queue_timeout`	排隊超過收件匣設定的「等待超時時間」仍無人接手，MaiAgent 自動把對話轉回 AI	`assignee` 為 `null`。真人客服在你自己的系統時，收到它代表 MaiAgent 已在你不知情下恢復 AI 回覆——請視為需要告警的異常；或在派工設定選「接手制」並勾選「不逾時」（API 為 `dispatchMode: "handover"`、`assignmentQueueTimeout: 0`；自動派工模式下送 0 會被改成 300 秒）

四種事件的 payload 鍵集、簽章與重試行為完全相同，接收端**依 `event` 欄位分流**；未來可能新增事件種類，遇到不認識的 `event` 請回 2xx 並忽略，不要當成錯誤。

`handoff.returned_to_ai` 與 `handoff.queue_timeout` 都代表「AI 恢復自動回覆」，差別在前者是人（專員或你的系統）主動、後者是逾時自動。兩者都不等於「對話已結案」：客服直接把對話標為「已解決」、或客服因失去收件匣權限而使對話重新排隊時，不會送出事件。需要完整的結案訊號時，請以對話 API 查詢 `status`，或在訊息 Webhook 的 `conversation.progress_status` 觀察狀態變化。

事件格式

```
POST <你設定的網址>
Content-Type: application/json
```

```
{
  "event": "handoff.assigned",
  "id": "9f0e1d2c-...",
  "occurred_at": "2026-09-09T02:30:00.123456+00:00",
  "conversation": {
    "id": "6c2b7c1e-...",
    "url": "https://admin.maiagent.ai/..."
  },
  "inbox": { "id": "0f9a3d42-..." },
  "organization": { "id": "b1d20c77-..." },
  "contact": { "id": "a4c1f6b3-..." },
  "assignee": { "id": "3e8f5a90-..." }
}
```

欄位	類型	說明
event	string	事件名稱（見上方「事件種類」）
id	string (uuid)	事件識別碼，與事件紀錄 API 回傳的 `id` 相同；重試送出的內容完全一樣，**以它去重**
occurred_at	string (date-time)	事件發生時間（ISO 8601）；推送不保證依序到達，**以它排序**
conversation.id	string (uuid)	對話識別碼
conversation.url	string (uri)	後台對話頁連結
inbox.id	string (uuid)	收件匣識別碼
organization.id	string (uuid)	組織識別碼
contact	object	null
assignee	object	null

欄位命名與巢狀結構和收件匣「訊息 Webhook」的 MaiAgent 標準格式一致（snake_case、關聯物件以 `{ "id" }` 巢狀），同一支接收端可以同時處理兩組 Webhook。Payload 只含識別資訊、事件時間與後台連結，**不含對話文字內容與轉真人理由**；需要對話內容時，請以識別碼呼叫對話與訊息 API 查詢。

簽章驗證（建議啟用）

在後台設定簽章密鑰後，每次推送會帶下列標頭：

```
X-MaiAgent-Signature-256: sha256=
```

值為以密鑰對 **原始 request body bytes** 計算的 HMAC-SHA256 十六進位摘要。驗證時務必使用收到的原始 bytes，不要先解析再重新序列化（欄位順序與空白差異會使摘要不一致）。

```
import hashlib
import hmac


def verify_signature(raw_body: bytes, header_value: str, secret: str) → bool:
    expected = 'sha256=' + hmac.new(secret.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)
```

密鑰為 write-only：儲存後不回顯，後台僅顯示「已設定」；欄位留空儲存代表維持既有密鑰，更換直接輸入新值，移除請使用「清除密鑰」操作。未設定密鑰時，推送不帶簽章標頭。

接收端最小範例

以下是一支可直接啟動的 Python（Flask）接收端：讀取原始 bytes 驗章、依 `event` 分流、先回 2xx 再做耗時處理。把 `HANDOFF_WEBHOOK_SECRET` 換成後台設定的簽章密鑰即可。

```python
import hashlib
import hmac
import os

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["HANDOFF_WEBHOOK_SECRET"]


def verify_signature(raw_body: bytes, header_value: str | None) -> bool:
    if not header_value:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), raw_body,
hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, header_value)


@app.post("/webhooks/handoff")
def handoff_webhook():
    raw = request.get_data()  # 原始 bytes，不要用 request.json 再序列化
    if not verify_signature(raw, request.headers.get("X-MaiAgent-Signature-
256")):
        abort(401)

    event = request.get_json(force=True)
    if already_processed(event["id"]):  # 重試會送完全相同的內容，以事件 id 去重
        return "", 204

    conversation_id = event["conversation"]["id"]
    if event["event"] == "handoff.requested":
        # AI 已停止回覆；真人客服在你的系統時，在這裡開單、派客服
        open_ticket(conversation_id, event["conversation"]["url"])
    elif event["event"] == "handoff.assigned":
        # 指派給 MaiAgent 後台的客服；轉接給另一位客服時會再收到一筆，assignee 會變
        assign_ticket(conversation_id, event["assignee"]["id"])
    elif event["event"] == "handoff.returned_to_ai":
        close_ticket(conversation_id)
    elif event["event"] == "handoff.queue_timeout":
        # MaiAgent 已自動恢復 AI 回覆；若你的客服仍在處理，這是需要告警的異常
        close_ticket(conversation_id, reason="queue_timeout")
    # 未知事件：回 2xx 忽略，不要當錯誤

    return "", 204  # 10 秒內回 2xx；耗時工作丟到背景


def already_processed(event_id):
```

```
    ...


def open_ticket(conversation_id, url):
    ...


def assign_ticket(conversation_id, assignee_id):
    ...


def close_ticket(conversation_id, reason=None):
    ...
```

```
HANDOFF_WEBHOOK_SECRET=你的密鑰 flask --app receiver run --port 8080
```

本機測試可用下列指令模擬一筆推送（簽章用同一把密鑰算）：

```
BODY='{"assignee":{"id":"3e8f5a90-0000-0000-0000-000000000004"},"contact":
{"id":"a4c1f6b3-0000-0000-0000-000000000005"},"conversation":{"id":"6c2b7c1e-
0000-0000-0000-
000000000001","url":"https://admin.maiagent.ai/conversations/index?
conversationId=6c2b7c1e-0000-0000-0000-
000000000001"},"event":"handoff.assigned","id":"9f0e1d2c-0000-0000-0000-
000000000006","inbox":{"id":"0f9a3d42-0000-0000-0000-
000000000002"},"occurred_at":"2026-09-09T02:30:00.123456+00:00","organization":
{"id":"b1d20c77-0000-0000-0000-000000000003"}}'
SIG=$(printf '%s' "$BODY" | openssl dgst -sha256 -hmac "你的密鑰" | awk '{print
$NF}')
curl -X POST http://localhost:8080/webhooks/handoff
  -H "Content-Type: application/json"
  -H "X-MaiAgent-Signature-256: sha256=$SIG"
  -d "$BODY"
```

回應與重試

接收端應在 10 秒內以 2xx 狀態碼回應，回應內容不拘。

逾時、連線失敗或非 2xx 回應時，系統會自動重試，最多 3 次、間隔 60 秒；重試送出的內容與原始推送完全相同（同一個 `id`），接收端請以事件 `id` 去重。事件各自以背景任務送出，`handoff.requested` 與緊接著的 `handoff.assigned` 可能不依序到達，請以 `occurred_at` 排序。

Webhook 推送失敗不影響轉真人主流程，也不影響通知中心與 Email 通知。

轉真人後由外部系統接手對話

若你的真人客服不在 MaiAgent 後台，而是在自己的客服系統回覆，可以把本頁的事件、對話平台的「訊息 Webhook」與訊息 API 組合起來，讓對話在轉真人期間雙向同步：收到 `handoff.requested` 後在你的系統開單，以訊息 Webhook 接收訪客訊息，以 `POST /api/v1/messages/outgoing/` 寫回客服回覆，處理完畢呼叫 `PATCH /api/v1/conversations/{id}/return-to-ai/` 交回 AI。完整流程、呼叫範例與注意事項見技術手冊第三方客服系統串接（含轉真人）：訪客的通道掛在你的系統時看情境 A，訪客在 MaiAgent 的對話平台、只有真人客服在你的系統時看情境 B。

串接流程

見技術手冊情境 B › 串接流程。

呼叫範例

見技術手冊情境 B › 呼叫範例。

注意事項

見技術手冊情境 B › 注意事項。

客戶端工具（Client Tools）

客戶端工具讓 AI 助理可以呼叫「在你的應用程式裡執行」的工具。你在每次呼叫對話 API 時宣告工具清單；模型判斷需要用到某個工具時，MaiAgent 不會自己執行，而是把「要呼叫哪個工具、帶什麼參數」回傳給你，由你的應用程式在本機執行後，再把結果送回對話，模型會依結果繼續回答或呼叫下一個工具。

適合只有你的應用程式才能執行的動作，例如查詢只在公司內網可達的系統、瀏覽器擴充功能操作目前開啟的網頁、App 存取使用者裝置上的資源。

使用情境

一家電商的工程師把 AI 助理接進公司內部的客服工作台，訂單系統只開放在內網。客服人員在工作台輸入「客人問訂單 A1024 什麼時候到貨」，工作台呼叫對話 API 時一併宣告了「查詢訂單」工具。模型回覆要查哪一筆訂單，工作台在內網查到出貨狀態後送回同一段對話，模型據此寫出回覆客人的說明。訂單系統不必對外開放，工作台只送出模型需要的資料。

使用條件

客戶端工具已對所有組織開放，不需要另外申請。要讓模型能呼叫，需同時符合：

AI 助理的 回答模式 為 Agent 模式
AI 助理使用的模型支援工具呼叫

> 不符合上述條件時，API 不會回傳錯誤，而是直接忽略你宣告的客戶端工具，模型會在沒有這些工具的情況下回答。若模型始終不呼叫你的工具，請先檢查助理的回答模式與模型。

運作流程

宣告工具並送出訊息

呼叫 `POST /api/v1/chatbots/{chatbotId}/completions/`，在 `message.clientTools` 帶入工具定義。

收到工具呼叫要求

模型決定呼叫你的工具時，回應的 `contentPayload.items` 會出現 `type` 為 `client_tool_call` 的項目，**本回合隨即結束**。

在你的應用程式執行工具

依 `name` 與 `input` 執行對應的工具。

把結果送回同一段對話

以同一個 `conversation` 再呼叫一次對話 API，把執行結果寫在 `message.content`，並**再次帶上 `clientTools`**。模型會接續回答；若還需要其他工具，會再回到步驟 2，直到回傳最終文字為止。

宣告工具

```
{
  "conversation": "<對話 ID，第一則訊息可省略>",
  "isStreaming": false,
  "message": {
    "content": "客人問訂單 A1024 什麼時候到貨？",
    "clientTools": [
      {
        "name": "get_order",
        "description": "依訂單編號查詢訂單的出貨狀態與預計到貨日",
        "inputSchema": {
          "type": "object",
          "properties": {
            "order": { "type": "string", "description": "訂單編號" }
          },
          "required": ["order"]
        }
      }
    ]
  }
}
```

欄位	類型	說明
name	string	工具名稱，僅限英數字、底線 `_`、連字號 `-`，長度 1–64，同一次請求內不可重複
description	string	工具用途說明，模型依此判斷何時使用，請寫清楚
inputSchema	object	工具參數的 JSON Schema，序列化後不可超過 4,000 字元

每次請求最多宣告 20 個客戶端工具。 工具清單只在該次請求有效，不會記在對話上。後續每一則訊息（包含送回工具結果的訊息）都要重新帶上 clientTools，否則模型在那一回合就無法再呼叫。

工具呼叫要求的格式

模型要求呼叫工具時，`contentPayload.items` 會包含如下項目，一次可能有多個：

```
{
  "type": "client_tool_call",
  "toolCallId": "call_abc123",
  "name": "get_order",
  "input": { "order": "A1024" },
  "startTimestamp": "2026-10-01T08:30:00.000000+00:00"
}
```

欄位	類型	說明
type	string	固定為 `client_tool_call`
toolCallId	string	null
name	string	要執行的工具名稱，對應你宣告的 `name`
input	object	模型帶入的參數，對應你宣告的 `inputSchema`
startTimestamp	string	null

非串流與串流的差異

非串流（`isStreaming: false`）：工具呼叫項目在回應本體（`done: true`）的 `contentPayload.items` 中。

串流（`isStreaming: true`）：工具呼叫項目會以獨立事件送出，該事件 `done` 為 `false`、`content` 為空字串、`contentPayload.items` 內含 `client_tool_call`。最後的 `done: true` 事件不會再重複這些項目，請在串流過程中收集。

> 回應的欄位名稱一律轉為 camelCase，input 內的參數名稱也會被轉換。例如你在 inputSchema 定義的 order_id，在 input 中會變成 orderId。建議參數名稱使用單一小寫英文單字（如 order、keyword），或在解析時同時接受兩種寫法。

送回執行結果

目前沒有專用的「提交工具結果」端點。請在**同一段對話**送出下一則一般訊息，把結果寫在 `message.content`：

```
{
  "conversation": "<同一個對話 ID>",
  "isStreaming": false,
  "message": {
    "content": "工具 get_order（toolCallId: call_abc123）執行結果：\n<查詢結果>",
    "clientTools": [ " ... 與前一次相同的工具定義 ... " ]
  }
}
```

一次多個工具呼叫：全部執行完後，把各工具的結果合併在同一則訊息送回。

寫上工具名稱與 `toolCallId`：後端不會用 `toolCallId` 比對，但寫在結果文字中能幫助模型對應是哪一次呼叫。

執行失敗或使用者拒絕：同樣用文字說明，例如「get_order 執行失敗：查無此訂單」或「使用者拒絕執行此操作」，模型會據此調整回答或改用其他方式。

結果內容長度：送回的內容會計入模型輸入，資料量大時請先截取需要的部分。

連續呼叫多個工具

每一次「模型要求呼叫 → 你送回結果」就是一個回合。模型可以在多個回合中連續呼叫工具，直到回應中不再出現 `client_tool_call`、改為回傳最終文字。建議在你的應用程式設定回合上限，避免異常情況下無限循環。

常見問題

- ▶ 模型一直沒有呼叫我的工具
- ▶ 送回結果後模型說無法使用工具
- ▶ 收到 400 錯誤

訊息 Webhook

在對話平台（Inbox）設定訊息 Webhook 後，每一則訊息都會由 MaiAgent 主動推送到你的系統，不需要輪詢。常見用途：在自己的系統顯示「有新訊息」、把客戶留言同步到 CRM、把 AI 回覆記錄到資料倉儲。

一個對話平台可以設定多組訊息 Webhook，每組各自選擇類型、網址與格式，彼此獨立推送。

> 只想在轉真人時收到通知，請用 轉真人 Webhook 事件：它只在轉真人流程的狀態改變時推送，不會每則訊息都推送。兩者是各自獨立的設定，可以同時啟用、由同一支接收端處理。

類型：Incoming 與 Outgoing

類型	何時推送	payload 的 `type`
`Incoming`	對話平台每收到一則客戶（聯絡人）訊息	`incoming`
`Outgoing`	每送出一則回覆到客戶所在的對話平台，包括 AI 自動回覆與真人客服的回覆	`outgoing`

後台 `Incoming` 選項的說明文字是「接收外部請求」，但實際行為是**把每則客戶訊息推送到你設定的網址**。

同一個選單中的 `Receive` 用途相反：由外部平台把訊息送進 MaiAgent，不屬於本篇的推送範圍。經由 `Receive` 送進來的訊息不會觸發同一對話平台的 `Incoming` ／ `Outgoing` 推送，避免訊息在兩個系統之間來回循環。

需要同時知道客戶訊息與回覆時，請各設一組 `Incoming` 與 `Outgoing`，可以指向同一個網址，再以 payload 的 `type` 區分。

設定方式

在後台設定

後台 → 客服對話 → 對話平台 → 選擇對話平台 → Webhook 設定 → 新增 Webhook

填寫 名稱，類型選 Incoming 或 Outgoing，填入 Webhook URL，選擇 HTTP Method（預設 `POST`）。

（建議）在 Headers 新增一組驗證用的標頭，例如 `Authorization: Bearer <你產生的隨機字串>`，接收端據此確認請求來自 MaiAgent（見驗證推送來源）。

Payload 格式選 MaiAgent Standard（見推送內容）；需要自訂欄位時選 自訂範本。

確認 啟用狀態 已開啟後儲存。關閉啟用狀態即暫停推送，設定會保留。

WhatsApp 對話平台的設定頁目前沒有 Webhook 設定 分頁，請改用下方的 API 設定。

以 API 設定

訊息 Webhook 掛在對話平台（Inbox）之下：

動作	方法與路徑
列出	`GET /api/v1/inboxes/{inbox_id}/webhooks/`
新增	`POST /api/v1/inboxes/{inbox_id}/webhooks/`
查詢單筆	`GET /api/v1/inboxes/{inbox_id}/webhooks/{id}`
修改	`PATCH /api/v1/inboxes/{inbox_id}/webhooks/{id}` （只送要改的欄位）
刪除	`DELETE /api/v1/inboxes/{inbox_id}/webhooks/{id}`

API 金鑰擁有者需能管理該對話平台，詳見 API 認證 → API 金鑰的權限範圍。

欄位	必填	說明
`name`	是	名稱，方便辨識
`direction`	是	`incoming` 或 `outgoing`
`url`	是	接收推送的網址，必須可從網際網路連線（建議 `https://`）；指向內部網段或本機位址的網址會被拒送
`method`	否	`POST`（預設）、`PUT` 或 `PATCH`
`headers`	否	每次推送都會帶上的自訂標頭，JSON 物件，例如 `{"Authorization": "Bearer ..."}`
`templatePreset`	否	`maiagent_standard`（預設）、`custom`，或串接 Genesys Cloud 用的 `genesys_cloud`
`customTemplate`	`custom` 時必填	Jinja2 範本，渲染結果必須是 JSON；儲存時會先試算一次，格式錯誤會回 400
`secretToken`	否	簽章密鑰；設定後每次推送都帶 HMAC-SHA256 簽章與時間戳標頭（見驗證推送來源）。只能寫入，查詢時不會回傳
`isEnabled`	否	是否啟用，預設 `true`

```
curl -X POST "https://api.maiagent.ai/api/v1/inboxes/YOUR_INBOX_ID/webhooks/"
  -H "Authorization: Api-Key YOUR_API_KEY"
  -H "Content-Type: application/json"
  -d '{
    "name": "新訊息通知",
    "direction": "incoming",
    "url": "https://example.com/webhooks/maiagent-message",
    "secretToken": "YOUR_RANDOM_SECRET"
  }'
```

成功回 `201` 與該筆設定（不含 `secretToken`）。要再收回覆，以 `"direction": "outgoing"` 再新增一筆。

推送內容

每則訊息以一個 HTTP 請求推送：

標頭：`Content-Type: application/json`、你設定的自訂標頭、`X-MaiAgent-Delivery-Attempt`（第幾次嘗試，第一次為 `1`），以及有設定 `secretToken` 時的 `X-MaiAgent-Timestamp`、`X-MaiAgent-Signature-256`。自訂標頭與這三個標頭同名（不分大小寫）時，送出前會被移除。

內容：依 Payload 格式 產生的 JSON。

MaiAgent Standard 格式

欄位一律為 snake_case（與 API 回應的 camelCase 不同）。以下是一則 `Outgoing`（AI 回覆）的推送內容：

```
{
  "id": "a0c49ae2-013c-4310-b00f-b4554bc9cd83",
  "type": "outgoing",
  "content": "您好，已收到您的訂單問題，請提供訂單編號。",
  "created_at": "2026-10-03T04:59:01.512340+00:00",
  "conversation": {
    "id": "e6b27044-4990-4129-ad8e-ac7649984e7d",
    "title": "訂單地址修改",
    "type": "individual",
    "status": "open",
    "progress_status": "ai_processing",
    "auto_reply_enabled": true,
    "assignee": null
  },
  "inbox": { "id": "6571672b-2fae-4200-a4d0-cf12e86216c3", "name": "官網客服",
"channel_type": "web" },
  "contact": { "id": "350691e0-66a5-48ea-855c-d8bb90a7e24a", "name": "王小明",
"email": null, "phone_number": null },
  "sender": { "id": "8a214fc3-75ad-43ba-93f6-567e9df97b76", "name": "客服助理",
"type": "chatbot" },
  "attachments": [],
  "metadata": {}
}
```


欄位	說明
`id`	訊息 id，可用來去重，也可用於訊息 API 查詢
`type`	`incoming`（客戶訊息）或 `outgoing`（回覆）；判斷方向請用這個欄位
`content`	訊息文字
`created_at`	訊息建立時間（ISO 8601，UTC）
`conversation`	所屬對話。`progress_status` 為 `human_serving` 表示目前由真人客服處理；`assignee` 是被指派的客服 `{ "id", "name" }`，未指派時為 `null`
`inbox`	對話平台 `{ "id", "name", "channel_type" }`，`channel_type` 例如 `web`、`line`、`messenger`
`contact`	客戶（聯絡人）；`email`、`phone_number` 沒有資料時為 `null`
`sender`	發話者，`type` 為 `contact`（客戶）、`chatbot`（Agent）或 `user`（組織成員，例如真人客服；以 API 金鑰代客戶送出的訊息也會是 `user`）

欄位	說明
`attachments`	附件陣列，每筆含 `id`、`type`（`image`／`video`／`audio`／`sticker`／`other`）、`filename`、`source_url`
`metadata`	訊息的附加資訊，內容依對話平台而異

自訂範本

自訂範本（API 為 `templatePreset: "custom"`）以 Jinja2 語法組出你要的 JSON。可用變數：`message`（`id`、`type`、`content`、`created_at`、`metadata`）、`conversation`、`inbox`、`contact`、`sender`、`sender_type`（`contact`／`chatbot`／`user`）、`attachments`、`now`。字串請用 `tojson` 過濾器輸出，避免引號或換行破壞 JSON：

```
{
  "text": {{ message.content | tojson }},
  "direction": "{{ message.type }}",
  "conversation_id": "{{ conversation.id }}",
  "customer": {{ (contact.name if contact else none) | tojson }}
}
```

推送當下範本渲染失敗時，該則訊息不會送出。

驗證推送來源

設定 `secretToken`（API 欄位，或後台 Webhook 表單的 Secret Token；Payload 格式為 Genesys Cloud 時後台不顯示此欄位）後，每次推送都會帶以下標頭：

標頭	內容
`X-MaiAgent-Timestamp`	送出當下的 Unix 時間（秒）
`X-MaiAgent-Signature-256`	`sha256=` 加上 `HMAC-SHA256(secretToken, "{timestamp}." + 原始 body)` 的十六進位值

接收端驗證步驟：

用收到的**原始 body 位元組**計算簽章，不要先解析成 JSON 再重新序列化（位元組會不同，簽章對不上）。

以固定時間比較（例如 Python 的 `hmac.compare_digest`）與 `X-MaiAgent-Signature-256` 比對。

`X-MaiAgent-Timestamp` 與接收端目前時間相差超過容許範圍（例如 5 分鐘）就拒絕，避免重放。每次嘗試（含重送）都依送出當下重新計算時間戳與簽章。

完整說明見技術手冊的驗證簽章與時間戳。

不使用簽章時，也可以在 `headers` 自訂驗證標頭（例如 `Authorization: Bearer <隨機字串>`）。`headers` 的值在查詢 Webhook 設定時會原樣回傳，能查看該對話平台設定的組織成員都看得到；`secretToken` 只能寫入、不會回傳。

設定 `secretToken` 時仍會另外送出舊的 `X-Webhook-Secret` 標頭（值為 `secretToken` 原文），供既有接收端相容；新串接請改驗簽章。

驗證用的密鑰請用夠長的隨機值，並透過 HTTPS 接收。

回應與重試

訊息寫入後才在背景推送，通常數秒內送達；不同訊息各自送出，**不保證到達順序**，請以 `created_at` 排序、以 `id` 去重。

接收端請在 **30 秒**內回應 2xx。逾時、連線失敗或非 2xx 都視為失敗。

暫時性失敗會自動重送：逾時、連線失敗、`5xx`、`429` 會在約 1、5、15 分鐘後各重送一次，最多重送 3 次（含第一次共 4 次嘗試）。其他 `4xx`、目的地被安全防護封鎖、自訂範本渲染失敗都不重送。重送的 body 與第一次完全相同，`X-MaiAgent-Delivery-Attempt` 依序為 `2`、`3`、`4`；間隔固定，不讀取回應的 `Retry-After`。重送前 Webhook 已停用或刪除、或訊息已刪除，就不再重送。細節見技術手冊的自動重送。

有了重送，推送是「至少送達一次」：接收端其實已處理、但回應逾時或回了 `5xx`，同一則訊息會再收到一次，請以訊息 `id` 去重。建議先把內容落地再回 2xx、耗時處理放到背景；確定永遠無法處理才回 `4xx`，暫時無法處理（例如下游服務故障）請回 `5xx` 或 `429`。

3 次重送都失敗就不再補送，訊息 Webhook **不保證送達**。偵測新訊息的輪詢做法可以低頻補查**客戶訊息**（Incoming）；AI 或客服的回覆（Outgoing）常在客戶訊息之後才產生，輪詢範例不一定會再讀到，不能靠它補回推送失敗的回覆。要找出推送失敗的訊息，請查下方的傳送紀錄：`statusCode` 不是 2xx 或為空（逾時、連線失敗）的就是失敗的推送，`requestBody` 是當時推送的內容（MaiAgent Standard 格式內有訊息 `id` 與 `conversation.id`），再讀該對話訊息列表的第一頁補上。

每次嘗試（含重送）的網址、內容、回應狀態碼都會各留一筆傳送紀錄，可用 `GET /api/v1/chatbots/{chatbot_id}/webhook-records/` 查詢（需具備該 Agent 的編輯權限；`chatbot_id` 為對話平台綁定的 Agent）。注意此 API 的回應同樣轉為 camelCase，實際推送的內容是 snake_case。

推送失敗不影響訊息本身的收發與 AI 回覆。

接收端最小範例

以下是一支可直接啟動的 Python（Flask）接收端：驗證簽章與時間戳、以訊息 `id` 去重（重送時同一則會再收到）、依 `type` 分流，先回 2xx 再處理。把 `MESSAGE_WEBHOOK_SECRET` 換成你設定的 `secretToken`；若改用自訂標頭驗證，改讀對應的標頭即可。

```python
import hashlib
import hmac
import os
import time

from flask import Flask, abort, request

app = Flask(__name__)
SECRET = os.environ["MESSAGE_WEBHOOK_SECRET"]
TOLERANCE_SECONDS = 300
processed_ids: set[str] = set()  # 正式環境請改用資料庫或 Redis，並設定保存期限


def is_valid_signature(timestamp: str, signature: str, body: bytes) -> bool:
    if not timestamp.isdigit() or abs(time.time() - int(timestamp)) >
TOLERANCE_SECONDS:
        return False
    expected = "sha256=" + hmac.new(SECRET.encode(), f"{timestamp}.".encode() +
body, hashlib.sha256).hexdigest()
    return hmac.compare_digest(signature, expected)


@app.post("/webhooks/maiagent-message")
def message_webhook():
    if not is_valid_signature(
        request.headers.get("X-MaiAgent-Timestamp", ""),
        request.headers.get("X-MaiAgent-Signature-256", ""),
        request.get_data(),
    ):
        abort(401)

    message = request.get_json(force=True)
    if message["id"] in processed_ids:
        return "", 204
    processed_ids.add(message["id"])

    conversation_id = message["conversation"]["id"]
    if message["type"] == "incoming":
        notify_new_message(conversation_id, message["content"])  # 例如顯示「有新訊
息」
    elif message["type"] == "outgoing":
        archive_reply(conversation_id, message["content"])

    return "", 204  # 30 秒內回 2xx；逾時或 5xx 會觸發重送，耗時工作請丟到背景


def notify_new_message(conversation_id, content):
```

```
    ...


def archive_reply(conversation_id, content):
    ...
```

```
MESSAGE_WEBHOOK_SECRET=你的密鑰 flask --app receiver run --port 8080
```

本機測試可用下列指令模擬一筆簽章正確的推送：

```
BODY='{"id":"acadbd9c-0000-0000-0000-000000000001","type":"incoming","content":"我
要改訂單地址","created_at":"2026-10-03T04:58:55.262828+00:00","conversation":
{"id":"e6b27044-0000-0000-0000-000000000002","title":"訂單地址修
改","type":"individual","status":"open","progress_status":"ai_processing","auto_re
ply_enabled":true,"assignee":null},"inbox":{"id":"6571672b-0000-0000-0000-
000000000003","name":"官網客服","channel_type":"web"},"contact":{"id":"350691e0-
0000-0000-0000-000000000004","name":"王小
明","email":null,"phone_number":null},"sender":{"id":"350691e0-0000-0000-0000-
000000000004","name":"王小明","type":"contact"},"attachments":[],"metadata":{}}'
TS=$(date +%s)
SIG=$(printf '%s.%s' "$TS" "$BODY" | openssl dgst -sha256 -hmac "你的密鑰" | sed
's/^.* //')
curl -X POST http://localhost:8080/webhooks/maiagent-message
  -H "Content-Type: application/json"
  -H "X-MaiAgent-Timestamp: $TS"
  -H "X-MaiAgent-Signature-256: sha256=$SIG"
  --data-raw "$BODY"
```

偵測新訊息

想在自己的系統顯示「有新訊息」的提醒、或在客戶留言時通知真人客服，建議依下列順序選擇做法：

優先用 Webhook 推送：對話平台的訊息 Webhook每則訊息推送一次，不需要輪詢（見用 Webhook 取代輪詢）。

只關心轉真人：用轉真人 Webhook 事件，只在轉真人流程的狀態改變時推送。

必須輪詢時：依本篇的輪詢建議做法，只找出有更新的對話、每段對話只讀訊息列表的第一頁。

> 請不要每輪把整段對話歷史從最新翻到最舊重抓。一段數百則訊息的對話每輪要讀好幾頁，每頁都是一次完整查詢；每隔幾秒對多段對話這樣做，會讓你的輪詢本身變慢，也會拖慢平台的整體回應速度。

用 Webhook 取代輪詢

需求	建議用法	何時推送
客戶每傳一則訊息就通知	對話平台的訊息 Webhook，類型選 `Incoming`	每則客戶（聯絡人）訊息
同步 AI 或客服送出的回覆	對話平台的訊息 Webhook，類型選 `Outgoing`	每則送往客戶所在對話平台的回覆
對話需要真人接手時通知	轉真人 Webhook 事件	只在轉真人、指派客服、轉回 AI、排隊逾時時推送，**不會**每則訊息都推送

設定位置：後台 → 客服對話 → 對話平台 → 選擇對話平台 → Webhook 設定 → 新增 Webhook，類型選 Incoming 或 Outgoing（後台 Incoming 的說明文字雖為「接收外部請求」，實際是把每則客戶訊息推送到你設定的網址；接收外部訊息的是 Receive），Payload 格式選 MaiAgent Standard（MaiAgent 標準格式）。推送內容、來源驗證與重試行為見訊息 Webhook。

WhatsApp 對話平台的設定頁目前沒有 Webhook 設定 分頁，請改用下方的輪詢做法。

兩種 Webhook 是各自獨立的設定，可同時啟用；推送內容都以訊息或事件的 `id` 去重。訊息 Webhook 遇到逾時、連線失敗、`5xx`、`429` 會自動重送最多 3 次（同一則可能收到不只一次），3 次都失敗就不再補送，不保證送達。下方的輪詢做法可以低頻補查**客戶訊息**（Incoming）；AI 或客服的回覆（Outgoing）推送失敗時，輪詢範例不一定會讀到，請改查 Webhook 的傳送紀錄找出失敗的推送（見訊息 Webhook → 回應與重試）。

輪詢建議做法

沒辦法接收 Webhook（例如接收端無法對外開放）時，請用下面兩個 API 組合輪詢。

1. 找出有更新的對話

```
GET /api/v1/conversations/?inbox=<對話平台 id>&page_size=20
```

對話列表固定**依最後訊息時間由新到舊**排列（回應中的 `lastMessageCreatedAt`）。每段對話記下上次看到的 `lastMessageCreatedAt` 與最後一則訊息的 `lastMessage.id`，兩者任一不同就是有更新的對話（時間只精確到秒，同一秒連傳兩則時只有 `id` 會變）；每輪從第一筆往下讀，讀到比「目前看過的最新時間往前 10 分鐘」還舊的對話就停。一般情況只需要讀第一頁；只有這段時間內更新的對話超過 `page_size` 段，才需要跟著回應的 `next` 讀下一頁。

往前多看 10 分鐘是因為：以 API 對話時，對話的 `lastMessageCreatedAt` 可能在 AI 回覆產生完才寫入，寫入的仍是客戶訊息的時間，所以有可能比你上一輪看過的其他對話還舊。只比較「是否比上輪最新時間新」會漏掉這類對話。

2. 每段有更新的對話只讀第一頁訊息

```
GET /api/v1/messages/?conversation=<對話 id>&page_size=20
```

訊息列表的**第一頁就是最新的訊息**：系統先取最新的 `page_size` 則，頁內再依時間由舊到新排列，所以最後一則就是整段對話最新的訊息。回應中的 `next` 指向**更舊**的訊息，偵測新訊息時不要每輪都往回翻。

以訊息 `id` 判斷是否處理過：第一頁裡沒處理過的 `id` 就是新訊息。只有一種情況需要翻頁——第一頁**全部**都是沒處理過的訊息（兩輪之間進來超過 `page_size` 則），才跟著 `next` 往更舊讀，讀到處理過的訊息為止。

3. 輪詢間隔與每頁筆數

設定	建議	說明
輪詢間隔	30–60 秒	需要更即時的通知請改用 Webhook，不要把間隔縮到幾秒
`page_size`	20	預設 20、上限 100。每頁筆數越大，單次回應越慢、內容越大

- 回應欄位為 camelCase；lastMessageCreatedAt、createdAt 是毫秒單位的時間戳記字串（例如 "1790753081000"），但只精確到秒——同一秒內可能有多則訊息，請以訊息 id 去重，不要只比時間。 - 範例以本機時間作為開始偵測的起點，執行的主機請開啟網路校時（NTP），避免時鐘偏快而略過剛開始時的訊息。 - 對話列表只列出有客戶訊息的對話；只有開場問候、客戶尚未發話的對話不會出現。 - 客戶傳來新訊息時 lastMessageCreatedAt 一定會更新，但可能晚於訊息本身寫入（見上方步驟 1）；AI 自動回覆不一定會更新它。因此本篇的輪詢做法只偵測客戶訊息：AI 回覆只有在讀取當下已產生才會一併讀到；客戶訊息讀過之後才產生的 AI 回覆，若沒有讓對話的最後訊息時間或最後一則訊息改變，之後的輪詢會略過這段對話，不會再讀到這則回覆。若單則 AI 回覆產生超過 10 分鐘（例如長時間執行工具），該則客戶訊息也可能延到對話下一次更新才被偵測。需要即時取得每則客戶訊息與回覆，請使用 Incoming／Outgoing 訊息 Webhook（暫時性失敗自動重送最多 3 次，仍不保證送達）。

呼叫範例

cURL

```
# 1. 依最後訊息時間由新到舊列出對話平台的對話
curl "https://api.maiagent.ai/api/v1/conversations/?
inbox=YOUR_INBOX_ID&page_size=20"
  -H "Authorization: Api-Key YOUR_API_KEY"

# 2. 只讀有更新的對話的第一頁訊息（最新 20 則，頁內舊→新）
curl "https://api.maiagent.ai/api/v1/messages/?
conversation=CONVERSATION_ID&page_size=20"
  -H "Authorization: Api-Key YOUR_API_KEY"
```

Python

```python
import os
import time

import requests

API = "https://api.maiagent.ai/api/v1"
HEADERS = {"Authorization": f"Api-Key {os.environ['MAIAGENT_API_KEY']}"}
INBOX_ID = os.environ["INBOX_ID"]
POLL_SECONDS = 30
PAGE_SIZE = 20
LOOKBACK_MS = 10 * 60 * 1000  # 往前多看 10 分鐘：對話的最後訊息時間可能在 AI 回覆產生後才寫入


def get(url: str, params: dict | None = None) -> dict:
    resp = requests.get(url, headers=HEADERS, params=params, timeout=10)
    resp.raise_for_status()
    return resp.json()


def recent_conversations(since_ms: int) -> list[dict]:
    """對話列表依最後訊息時間由新到舊排列：讀到比 since_ms 舊的就停，通常只讀第一頁。"""
    conversations = []
    url, params = f"{API}/conversations/", {"inbox": INBOX_ID, "page_size": PAGE_SIZE}
    while url:
        page = get(url, params)
        for conv in page["results"]:
            if int(conv["lastMessageCreatedAt"]) < since_ms:
                return conversations
            conversations.append(conv)
        url, params = page["next"], None
    return conversations


def new_messages(conversation_id: str, seen_ids: set[str], not_before_ms: int) -> list[dict]:
    """讀第一頁（最新的訊息，頁內由舊到新）；只有整頁都沒處理過時，才往更舊的頁讀。"""
    found = []
    url, params = f"{API}/messages/", {"conversation": conversation_id, "page_size": PAGE_SIZE}
    while url:
        page = get(url, params)
        fresh = [m for m in page["results"] if m["id"] not in seen_ids and int(m["createdAt"]) >= not_before_ms]
        found = fresh + found
        if len(fresh) < len(page["results"]):
```

```
            break  # 這一頁已碰到處理過（或開始偵測前）的訊息，不必再往回翻
        url, params = page["next"], None
    return found


def change_key(conv: dict) -> tuple:
    """時間只精確到秒：同一秒連傳兩則時只有最後一則訊息的 id 會變，所以一起比對。"""
    return conv["lastMessageCreatedAt"], (conv.get("lastMessage") or
{}).get("id")


def main() -> None:
    started_ms = int(time.time()) * 1000  # 從現在（取整到秒）開始偵測，不回頭處理歷史訊
息
    newest_ms = started_ms
    last_seen: dict[str, tuple] = {}  # 每段對話上次看到的 change_key（正式環境請持久化
或定期清理）
    seen_ids: dict[str, set[str]] = {}  # 每段對話已處理過的訊息 id（同上）
    while True:
        try:
            round_newest = newest_ms
            for conv in recent_conversations(newest_ms - LOOKBACK_MS):
                if last_seen.get(conv["id"]) == change_key(conv):
                    continue  # 這段對話沒有新的客戶訊息（之後才產生的 AI 回覆不會在這裡讀
到）
                ids = seen_ids.setdefault(conv["id"], set())
                for msg in new_messages(conv["id"], ids, started_ms):
                    print(f"[新訊息] 對話 {conv['id']} {msg['type']}：
{msg['content'][:40]}")
                    ids.add(msg["id"])
                last_seen[conv["id"]] = change_key(conv)
                round_newest = max(round_newest,
int(conv["lastMessageCreatedAt"]))
            newest_ms = round_newest  # 整輪成功才前進
        except requests.RequestException as exc:
            print(f"本輪查詢失敗，下一輪重試：{exc}")  # 回看起點沒有前進，未處理完的對話下
一輪會補上
        time.sleep(POLL_SECONDS)


if __name__ == "__main__":
    main()
```

API 金鑰的權限與金鑰擁有者在組織中的角色相同，擁有者需能查看該對話平台的對話，詳見 API 認證 → API 金鑰的權限範圍。

解析器自動偵測結果

上傳檔案到知識庫時不指定 `parser`（解析器留在「自動偵測（預設）」），系統會依檔案內容替這份檔選解析器。列出知識庫中的檔案與取得特定檔案詳情的回應中，每份檔案的 `parserAutoDetection` 會帶出這次判斷選了哪個解析器、為什麼，以及判斷時算出的數字。可以用它在自己的系統裡區分「系統自動選的解析器」與「使用者自己選的解析器」，並顯示選擇的原因。

欄位說明

欄位	類型	說明
`parser`	object	自動偵測實際選到、這份檔正在使用的解析器，欄位與檔案回應的 `parser` 相同（`id`、`name`、`provider`、`isTimestampSttProvider`）
`reason`	string	原因類別，見原因類別
`facts`	object	判斷時算出的數字，見判斷數字；沒有數字時為空物件 `{}`
`detectedAt`	string	判斷時間，Unix 時間戳記的毫秒字串（見欄位格式）

`facts` 的數字是判斷當下算出、之後不重算。用同一個解析器重新解析時不會重新判斷，數字與判斷時間維持原本那一次。

何時為 null

`parserAutoDetection` 為 `null` 代表這份檔目前用的解析器不是自動偵測選的，或還沒有判斷結果：

上傳或重新解析時指定了解析器（使用者自己選的）。

用「批量重新解析檔案」把自動偵測選的解析器改成別的解析器。用同一個解析器重新解析則保留原本的結果。

用「批量重新解析檔案」改回自動偵測（該檔的項目明確帶 `"parser": null`），新的判斷還在排隊、尚未完成。判斷完成後會換成新的結果。

這項功能上線前上傳的檔案（舊檔不回填）。

> 用批量重新解析檔案改回自動偵測時，該檔的項目要明確帶 "parser": null。項目裡沒有 parser 這個鍵時，這份檔不會更換解析器、也不會重新解析，parserAutoDetection 維持原樣。上傳時則可以直接省略 parser。

回應範例

PDF 檔掃描頁多，系統選了 Vision 解析器：

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "filename": "合約掃描檔.pdf",
  "parser": {
    "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
    "name": "Vision Parser",
    "provider": "anthropic",
    "isTimestampSttProvider": false
  },
  "parserAutoDetection": {
    "parser": {
      "id": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
      "name": "Vision Parser",
      "provider": "anthropic",
      "isTimestampSttProvider": false
    },
    "reason": "scanned",
    "facts": {
      "analyzedPages": 50,
      "partial": true,
      "nonBlankPages": 48,
      "scannedPages": 40,
      "imagePages": 42
    },
    "detectedAt": "1760860800000"
  }
}
```

使用者上傳時自己選了解析器：

```
{
  "id": "550e8400-e29b-41d4-a716-446655440001",
  "filename": "產品手冊.docx",
  "parserAutoDetection": null
}
```

範例只列出相關欄位，其他檔案欄位見各端點的回應結構。

原因類別

`reason`	意思	`facts`
`scanned`	內容以掃描頁為主（PDF 或 Word），需要以圖片判讀	PDF 或 Word 的數字
`image_pages`	PDF 有夠多需要以圖片判讀的頁（掃描頁或其他以圖片為主的頁）	PDF 的數字
`large_images`	Word 內文有夠多大圖	Word 的數字
`unmeasurable_images`	Word 內文有圖片，但量不到圖片尺寸，以需要圖片判讀處理	Word 的數字，可能為空物件
`presentation`	簡報檔（PPTX）	空物件
`no_image_content`	沒有需要以圖片判讀的頁，也沒有內文大圖	PDF 或 Word 的數字；Word 檔完全沒有圖片時為空物件
`below_threshold`	有需要以圖片判讀的頁或內文大圖，但未達選用 Vision 解析器的門檻	PDF 或 Word 的數字
`no_content`	PDF 判斷的頁全部是空白頁	PDF 的數字
`file_type_not_detected`	這種檔案類型不做內容判斷（例如舊版簡報 PPT），系統以該類型的預設解析器處理	空物件
`detection_failed`	判斷過程出錯，系統以該檔案類型的預設解析器處理	空物件
`vision_unavailable`	內容需要以圖片判讀，但沒有可用的 Vision 解析器；`parser` 是實際使用的解析器	與內容類別相同（PDF 或 Word 的數字；PPTX 為空物件）

> 判斷規則之後可能新增原因類別。請不要只依上表寫死判斷：遇到不認得的 reason 時，請顯示通用說明（例如「系統依檔案內容自動選擇了解析器」），不要報錯或略過整筆檔案。

判斷數字

`facts` 的鍵依檔案類型不同。依上表，PPTX、完全沒有圖片的 Word 檔、不做內容判斷的檔案類型與判斷出錯時為空物件 `{}`。

PDF

鍵	類型	說明
`analyzedPages`	integer	判斷時看了幾頁

鍵	類型	說明
`partial`	boolean	是否只看了前面幾頁（檔案頁數超過判斷上限，目前為前 50 頁時為 `true`）
`nonBlankPages`	integer	看過的頁中非空白的頁數
`scannedPages`	integer	看過的頁中的掃描頁數
`imagePages`	integer	看過的頁中需要以圖片判讀的頁數（含掃描頁）

Word（DOCX）

鍵	類型	說明
`figures`	integer	內文大圖的數量
`textChars`	integer	內文字數

Word 檔完全沒有圖片（`no_image_content`）或量不到圖片尺寸（`unmeasurable_images`）時，`facts` 可能為空物件。讀取時請把每個鍵都當成可能不存在。